



Carrera:
Programación y WebMaster

Docente:
MTRO. Mario Alonso Segovia
Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Salomon Campos Pablo Leonel

Fecha:
20 de Julio 2026

1. Introducción

La calidad de un sistema no solo depende de que funcione, sino también de que esté bien organizado y sea fácil de mantener. Cuando no existen reglas claras ni una buena forma de trabajar, los errores aumentan y cualquier cambio puede causar nuevos problemas.

En GymManager se encontraron varias fallas después de ocho meses de desarrollo. Por ello, en este trabajo se analizarán esos problemas y se propondrán mejoras relacionadas con Clean Code, SOLID, deuda técnica, MVP y Definition of Done.

2. Identificación de problemas

1. Clases con más de 1,800 líneas de código

¿Qué ocurre? Algunas clases tienen demasiado código y realizan varias tareas al mismo tiempo.

¿Por qué es una mala práctica? Porque se vuelven difíciles de entender, revisar y modificar. También muestra que las funciones del sistema no están bien separadas.

Consecuencias. Corregir errores toma más tiempo, se pueden afectar otras partes del sistema y resulta complicado que otro programador entienda el código.

2. Funciones con más de 250 líneas

¿Qué ocurre? Hay funciones demasiado largas que incluyen muchos procesos dentro del mismo bloque.

¿Por qué es una mala práctica? Una función debería realizar una tarea específica. Cuando contiene demasiadas instrucciones, su funcionamiento se vuelve confuso.

Consecuencias. Los errores son más difíciles de encontrar, las pruebas se complican y un pequeño cambio puede afectar todo el proceso.

3. Código reutilizado mediante copiar y pegar

¿Qué ocurre? Se copia el mismo código en diferentes módulos en lugar de crear una función que pueda reutilizarse.

¿Por qué es una mala práctica? Porque cuando se necesita hacer un cambio, hay que modificar todas las copias por separado.

Consecuencias. Puede haber resultados diferentes entre módulos, repetirse los mismos errores y aumentar el tiempo de mantenimiento.

4. Nombres de variables poco descriptivos

¿Qué ocurre? Se usan nombres como x, dato, temp, valor1 u obj, que no explican claramente qué guardan.

¿Por qué es una mala práctica? Porque dificultan entender el código y obligan a revisar más líneas para saber para qué sirve cada variable.

Consecuencias. Se pueden cometer errores al usar las variables y el equipo tarda más en revisar o modificar el código.

5. Un archivo realiza demasiadas tareas

¿Qué ocurre? Un mismo archivo se encarga de validar datos, consultar la base de datos, generar reportes y enviar correos.

¿Por qué es una mala práctica? Porque mezcla funciones diferentes en un solo lugar y hace que el archivo sea más difícil de manejar.

Consecuencias. Un cambio pequeño puede afectar otras funciones, las pruebas se complican y el código es más difícil de reutilizar.

6. Cada desarrollador organiza el proyecto de forma diferente

¿Qué ocurre? Cada integrante acomoda los archivos y clases según su propio criterio.

¿Por qué es una mala práctica? Porque no existe una estructura común que facilite encontrar y organizar el contenido del proyecto.

Consecuencias. Se pierde tiempo buscando archivos, pueden aparecer elementos repetidos y resulta más difícil integrar el trabajo de todo el equipo.

7. No existe una guía de estilos de programación

¿Qué ocurre? El equipo no tiene reglas comunes para escribir y organizar el código.

¿Por qué es una mala práctica? Porque cada desarrollador trabaja de manera distinta y el proyecto pierde uniformidad.

Consecuencias. El código termina con estilos mezclados, cuesta más revisarlo y se pierde tiempo corrigiendo detalles de formato y organización.

8. Las funcionalidades se aceptan cuando “parecen funcionar”

¿Qué ocurre? Una función se considera terminada solo porque funciona en una prueba básica.

¿Por qué es una mala práctica? Porque no se comprueba si cumple todo lo solicitado ni si responde bien ante errores o datos incorrectos.

Consecuencias. Pueden entregarse funciones incompletas, aparecer errores después y ser necesario repetir parte del trabajo.

9. No se realizan revisiones de código

¿Qué ocurre? Los cambios se agregan al proyecto sin que otro integrante los revise.

¿Por qué es una mala práctica? Porque una segunda revisión puede encontrar errores o mejoras que el autor no notó.

Consecuencias. Se integran fallas con mayor facilidad, se repiten malas prácticas y corregirlas después puede tomar más tiempo.

10. Cada nueva funcionalidad requiere modificar varias clases

¿Qué ocurre? Para agregar una función nueva, se tienen que cambiar varias clases que ya estaban hechas.

¿Por qué es una mala práctica? Porque muestra que el sistema no está preparado para crecer de forma sencilla.

Consecuencias. Un cambio pequeño puede provocar errores en otras partes, tardar más de lo esperado y complicar el mantenimiento.

11. Se agregan funciones que el cliente no pidió

¿Qué ocurre? El equipo desarrolla características que considera útiles, aunque no fueron solicitadas.

¿Por qué es una mala práctica? Porque se utiliza tiempo y dinero en funciones que no son prioridad.

Consecuencias. El proyecto puede retrasarse, aumentar su costo y terminar con herramientas que el cliente nunca utilice.

12. Se mantienen errores conocidos para avanzar más rápido

¿Qué ocurre? El equipo continúa trabajando aunque sabe que existen fallas pendientes.

¿Por qué es una mala práctica? Porque esos errores pueden afectar las nuevas funciones y hacerse más difíciles de corregir con el tiempo.

Consecuencias. Las fallas se acumulan, el sistema se vuelve inestable y el costo de reparación puede aumentar.

3. Análisis de Clean Code

1. Clases demasiado grandes

Principio incumplido. Las clases deberían ser pequeñas y encargarse de una tarea principal.

Cómo corregirlo. Se pueden separar las funciones en diferentes clases, por ejemplo, una para validar datos, otra para generar reportes y otra para enviar correos.

Beneficios. El código sería más fácil de entender, modificar y probar.

2. Funciones demasiado largas

Principio incumplido. Una función debe realizar una tarea específica y ser fácil de leer.

Cómo corregirlo. Las funciones largas deben dividirse en otras más pequeñas, como validarCliente, registrarPago o enviarConfirmacion.

Beneficios. Los procesos serían más claros y los errores podrían encontrarse con mayor facilidad.

3. Código duplicado

Principio incumplido. Se debe evitar repetir el mismo código en diferentes partes del sistema.

Cómo corregirlo. La lógica repetida puede colocarse en una función o clase que pueda utilizarse desde varios módulos.

Beneficios. Los cambios se realizan una sola vez, se reducen errores y el mantenimiento es más rápido.

4. Nombres poco claros

Principio incumplido. Los nombres deben explicar claramente para qué sirve cada variable o función.

Cómo corregirlo. Se deben usar nombres como cliente, montoPago, membresiaActiva o fechaAsistencia, en lugar de nombres como x o dato.

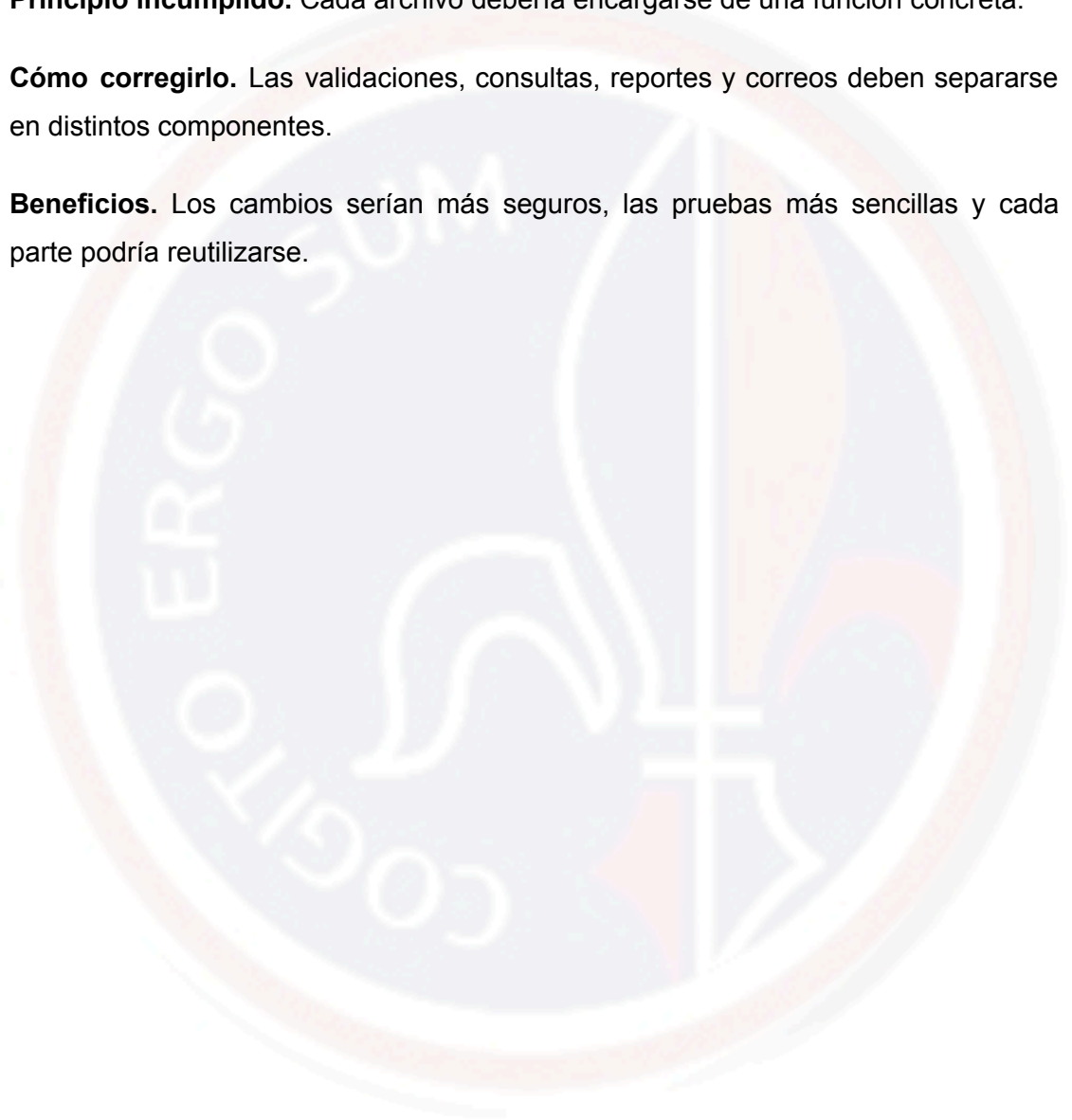
Beneficios. El código se entiende más rápido y se evita confundir la información utilizada.

5. Archivo con muchas responsabilidades

Principio incumplido. Cada archivo debería encargarse de una función concreta.

Cómo corregirlo. Las validaciones, consultas, reportes y correos deben separarse en distintos componentes.

Beneficios. Los cambios serían más seguros, las pruebas más sencillas y cada parte podría reutilizarse.



4. Aplicación de los principios SOLID

1. Principio de Responsabilidad Única (SRP)

¿En qué consiste? Cada clase debe encargarse de una sola tarea principal.

Evidencia en el caso. Un mismo archivo realiza validaciones, consultas, reportes y envío de correos. Además, existen clases demasiado extensas.

Cómo corregirlo. Se deben separar estas tareas en diferentes componentes, como validadores, servicios de reportes, consultas y notificaciones.

Beneficios. El código sería más ordenado, fácil de probar y sencillo de modificar.

2. Principio Abierto/Cerrado (OCP)

¿En qué consiste? El sistema debe permitir agregar nuevas funciones sin modificar constantemente el código que ya existe.

Evidencia en el caso. Cada nueva funcionalidad obliga a cambiar varias clases existentes.

Cómo corregirlo. Se pueden crear componentes que permitan agregar nuevas opciones, como tipos de reportes o notificaciones, sin alterar las clases principales.

Beneficios. Se reduciría el riesgo de causar errores y sería más fácil ampliar el sistema.

3. Principio de Inversión de Dependencias (DIP)

¿En qué consiste? La lógica principal no debe depender directamente de herramientas específicas, como el correo o la base de datos.

Evidencia en el caso. Las consultas, los reportes y los correos están unidos dentro del mismo archivo.

Cómo corregirlo. Se deben separar estas funciones y conectarlas mediante interfaces o servicios independientes.

Beneficios. El sistema sería más flexible, fácil de probar y permitiría cambiar una herramienta sin afectar todo el código.

5. Análisis de la deuda técnica

¿Qué tipo de deuda técnica observas en el proyecto?

Se observa deuda de código, diseño, pruebas y documentación. También es deuda deliberada, porque el equipo conoce algunos errores y decide continuar sin solucionarlos.

¿Cómo afectará al mantenimiento del sistema?

El mantenimiento será más complicado, ya que entender y modificar el código tomará más tiempo. Además, una corrección podría afectar otras partes del sistema.

¿Qué impacto tendrá sobre el tiempo y costo del desarrollo?

Aunque al principio permite avanzar más rápido, después aumentará el retrabajo. Esto provocará retrasos y mayores costos para corregir errores y realizar cambios.

¿Qué acciones propondrías para disminuir dicha deuda?

Se deben ordenar los problemas por importancia, corregir primero los más graves y mejorar el código poco a poco. También se necesitan pruebas, revisiones de código, reglas de programación y documentación actualizada.

6. Definición del Producto Mínimo Viable (MVP)

El MVP debe incluir únicamente las funciones necesarias para que el gimnasio pueda trabajar y llevar un control básico de sus actividades.

Funcionalidades que formarían parte del MVP

- **Gestión de clientes.** Permite registrar y consultar la información de las personas inscritas en el gimnasio. Es necesaria para relacionar pagos, asistencias y clases.
- **Registro de pagos.** Ayuda a controlar quién ha pagado su membresía y cuánto dinero ha recibido el gimnasio.
- **Control de asistencia.** Permite registrar la entrada de los clientes y conocer con qué frecuencia utilizan el servicio.
- **Reservación de clases.** Sirve para organizar horarios, lugares disponibles y personas inscritas en cada clase.
- **Reportes básicos de ingresos.** Permiten consultar los pagos recibidos durante un periodo y tener una idea general de los ingresos.

Funcionalidades para futuras versiones

- **Chat interno entre entrenadores.** Puede ser útil para comunicarse, pero no es necesario para el funcionamiento principal del gimnasio.
- **Integración con relojes inteligentes.** Requiere trabajar con dispositivos y servicios externos, por lo que su desarrollo sería más complicado.
- **Sistema de recompensas.** Puede motivar a los clientes, pero primero deben funcionar correctamente los módulos básicos.
- **Publicación automática en redes sociales.** Ayudaría a promocionar el gimnasio, aunque no forma parte de su administración diaria.
- **Tienda en línea de suplementos.** Necesitaría catálogo, inventario, pedidos y pagos en línea, lo que aumentaría mucho el tiempo de desarrollo.

7. Definition of Done (DoD)

Una función de GymManager se considerará terminada cuando cumpla con los siguientes puntos:

1. Cumple con lo solicitado por el cliente y con los requisitos establecidos.
2. Funciona correctamente de inicio a fin sin presentar errores importantes.
3. Valida los datos ingresados y muestra mensajes claros cuando existe algún error.
4. El código utiliza nombres fáciles de entender y mantiene una estructura ordenada.
5. No contiene código repetido que pueda reutilizarse mediante otra función o componente.
6. Se realizaron pruebas para comprobar los casos principales, datos incorrectos y posibles situaciones especiales.
7. Se verificó que funcione correctamente con la base de datos y con los módulos relacionados.
8. Otro integrante del equipo revisó el código y se corrigieron las observaciones encontradas.
9. La documentación relacionada con la función fue actualizada.
10. La función se integró correctamente al proyecto y se comprobó que no afecte otras partes del sistema.

8. Conclusión

Considero importante establecer reglas claras para saber cuándo una función está terminada. Ignorar estas prácticas puede generar errores, retrasos y mayores costos.

En mi proyecto aplicaré nombres claros, pruebas, documentación y una mejor organización para facilitar su mantenimiento.

